Lesson notes · for you

Threat-model an AI application

AI security · University · Lesson 1

Time	Materials
20 min	Paper and pens, or a whiteboard; An AI app you use or are building (looked at only as a user); The six STRIDE questions (on the slides)

Learning goals

- Draw an AI app's data flows and trust boundaries.
- Find threats at each boundary with STRIDE, and rank them.
- Shrink the blast radius by design.

Activity: Threat-model in twenty minutes

1. Draw (6 min): On one page, draw every part of the app you chose (the user, the app, the model, its tools, its documents, the outside content it reads) and an arrow for each data flow between them.
2. Boundaries (4 min): Circle each place where data you don't control enters: messages, uploads, web pages, tool results. Remember the model is not a boundary: anything in its context can steer it.
3. STRIDE (6 min): For each letter, write one realistic threat at your boundaries and score it likelihood × impact, 1–3 each. Mark every path from untrusted text to a part that acts.
4. Fix (4 min): Choose the one design change that shrinks the blast radius most (remove a tool, read-only access, approval in code) and say how you would test it.

Use an app you are building, or a made-up design. Never probe someone else's live system.

Discussion

- Why isn't the model a trust boundary?
- Which STRIDE letter was hardest to find a threat for?
- What can your app do that its job doesn't need?

Check yourself

- Name one path from untrusted text to an action in your app, and the change that would cut it.

Try it this week

With two classmates, draw the data flows of an AI app you use for your studies, circle every trust boundary, ask the six STRIDE questions, and agree on the one change that would shrink its blast radius most.

Worksheet

Threat-model an AI application

AI security · University · Lesson 1

Name _____ Class _____ Date _____

1 **The course assistant only needs to answer questions from the notes. Which change shrinks its blast radius the most?**

◯ A warning in the system prompt ◯ Removing the email tool ◯ A bigger, smarter model

2 **Match each STRIDE threat to what it could look like in the course assistant.**

Spoofing	• • • •	A web page gets it to send email
Tampering	• • • •	It reveals another student's marks
Information disclosure	• • • •	Someone edits the course notes it reads
Elevation of privilege	• • • •	A message claims to come from the lecturer

3 **The course assistant's threat model lists its data flows. Finish at_risk(flows, untrusted, acts): follow the flows hop by hop from the untrusted parts, and return the acting parts they can reach, sorted. The starter looks only one hop away, so it misses the path from web to model to email.**

```
FLOWS = [("model", "email"), ("web", "model"),
         ("student", "model"), ("model", "search")]

def at_risk(flows, untrusted, acts):
    reach = set(untrusted)
    for a, b in flows:
        if a in untrusted:
            reach.add(b)
    return sorted(c for c in acts if c in reach)

for c in at_risk(FLOWS, {"web"}, {"email"}):
    print("at risk:", c)
```

Write your program:

4 **The course assistant runs its tool calls with this code. Which STRIDE threat does it leave most open?**

```
def run_tool(call):
    if call["tool"] == "email":
        send(call["to"], call["body"])
    return "done"
```

- ◯ Repudiation: nothing records who sent what
- ◯ Denial of service: the code is too slow
- ◯ Spoofing: it checks a password

5 **Put the threat-modelling steps in order.**

- ☐ Give the top ones a control and a test
- ☐ Rank the threats by likelihood × impact
- ☐ Ask the STRIDE questions at each one
- ☐ Mark the trust boundaries
- ☐ Draw the data flows

✎ **Think of an AI app you use for your studies. Where does untrusted text get into it, and what is the worst thing it could be made to do?**